GYMHUB | GUION DE 20 MINUTOS

SCRUM-88 · HIA 2026 · FI - UNJu · 13/11/2026

5 integrantes, 4 minutos por persona. Guion terminado y listo para la exposición. Las partes de infraestructura que no se probaron se presentan como pendientes, no como logros demostrados.

Estado del material: guion, diapositivas y plan B preparados para presentar sin improvisar. **Duración total:** 20 minutos, 5 integrantes, 4 minutos por persona. **Fecha de corte de evidencias:** 13/11/2026. **Rama:** `docs/SCRUM-88-presentacion` → PR contra `develop`.

IMPORTANTE. No confundir «guion listo» con «todos los servicios del trabajo integrador funcionando». El Compose completo, la URL pública, la réplica/failover, Mantis y el informe IA en Nextcloud dependen de las tareas del equipo. Este guion describe qué mostrar si están operativos y cómo presentar honestamente lo que todavía no está integrado.

Reparto de tiempos y diapositivas

Turno	Integrante	Tiempo	Diapositivas	Qué muestra
1	Francisco Paredes	00:00–04:00	1–3	Introducción, alcance y arquitectura
2	Enzo Villagrán	04:00–08:00	4–6	Roles y demostración funcional
3	Valentín Lozano	08:00–12:00	7–8	Persistencia de una edición, BD y despliegue
4	Valentín Iriarte	12:00–16:00	9–10	Git/Jira, decisiones, seguridad y límites
5	Tiziano	16:00–20:00	11–14	Grafana, Prometheus, Alertmanager, n8n y cierre

La asignación está propuesta según los integrantes identificados en Jira; el grupo puede intercambiar partes, conservando las duraciones.

Guion completo por integrante

1. Francisco Paredes — 00:00 a 04:00 — Diapositivas 1, 2 y 3

00:00–00:20 | Diapositiva 1. «Buenas, somos el equipo de GymHub y vamos a mostrar el trabajo integrador de HIA. La idea es explicar cómo armamos el sistema, qué funciones pudimos probar y qué falta integrar. Vamos a usar la aplicación y los servicios de monitoreo que tenemos funcionando en el laboratorio».

00:20–01:40 | Diapositiva 2. «GymHub es una plataforma web para gestionar un gimnasio en un solo lugar. Antes podríamos tener usuarios, rutinas, cuotas y pagos repartidos en distintas herramientas. Acá la idea es centralizarlos. Hay tres roles: el administrador gestiona usuarios y pagos; el entrenador trabaja con ejercicios y rutinas; y el socio consulta sus actividades y sus cuotas. Esto es el alcance funcional del proyecto; después vamos a diferenciar qué pantallas y operaciones ya verificamos en vivo».

01:40–04:00 | Diapositiva 3. «La aplicación está desarrollada con Angular en el frontend y una API en Node.js y Express. La capa de persistencia usa Sequelize con PostgreSQL 16. En la instalación que probamos, el frontend está servido por Nginx dentro de Docker, en el puerto 8081; el backend funciona desde el host, en el puerto 3000; y PostgreSQL corre en otro contenedor. El navegador habla con el frontend y este hace de intermediario con la API. También agregamos Prometheus y Grafana para monitorear disponibilidad y

recursos, Alertmanager para gestionar alertas, y una instancia de n8n con datos persistentes. El objetivo final es levantar toda la aplicación con un Compose integrado, pero ese PR está en revisión. Hoy no vamos a presentar esa integración como terminada. Vamos a ver el sistema tal como fue comprobado».

Transición: «Ahora mostramos cómo cambia la aplicación según el rol que inicia sesión».

2. Enzo Villagrán — 04:00 a 08:00 — Diapositivas 4, 5 y 6

04:00–05:10 | Diapositiva 4. «El sistema distingue administrador, entrenador y socio. Las pantallas tienen menús diferentes según el usuario. Esto permite que un socio no tenga las mismas opciones que un administrador. La aplicación tiene guards de Angular, y la API también prevé controles de acceso por rol. La existencia de controles está en el código; en esta demo probamos principalmente el acceso a cada vista».

05:10–06:40 | Diapositiva 5. «Probamos el inicio de sesión con los tres usuarios de demostración. Con administrador aparecen usuarios, suscripciones, cuotas, pagos y panel de control. Con entrenador aparecen ejercicios y rutinas. Con socio aparecen mis rutinas y mis pagos. Estas tres capturas corresponden al frontend Docker, en localhost:8081, y son una evidencia de que la navegación por rol está funcionando». **Acción en pantalla (si todo responde):** abrir `http://localhost:8081/login` y mostrar un rol (preferiblemente entrenador). Para los otros dos, las capturas bastan si el tiempo es limitado. No dejar una contraseña visible.

06:40–08:00 | Diapositiva 6. «Como prueba funcional, entramos como entrenador y modificamos una rutina existente. Le pusimos el nombre “Prueba SCRUM-54.”. La pantalla mostró “Rutina modificada”. Lo importante no fue solo el cartel: después confirmamos el cambio consultando directamente PostgreSQL. Así verificamos que la interfaz, la API y la base se comunican». **Acción:** mostrar `http://localhost:8081/rutinas` y ubicar la rutina; usar la captura si no hay tiempo. No volver a modificar registros innecesariamente.

Transición: «Ahora vemos cómo comprobamos ese cambio desde la base de datos».

3. Valentín Lozano — 08:00 a 12:00 — Diapositivas 7 y 8

08:00–10:00 | Diapositiva 7. «Para comprobar el guardado hicimos una consulta de solo lectura a la tabla `rutinas`. El resultado fue una fila con ID 1, cuyo nombre es “Prueba SCRUM-54.”, igual a lo que se ve en la interfaz. Esto demuestra que una actualización desde el frontend llegó a PostgreSQL y luego pudo leerse. Es importante aclarar que modificamos una rutina que ya existía: esta consulta no prueba un alta nueva, ni que el dato siga después de recrear los contenedores». **Acción opcional de Terminal, solo lectura:**

```
docker exec demo_hia_db sh -lc 'psql -X -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT id, nombre,
descripcion FROM rutinas ORDER BY id DESC LIMIT 10;"'
```

10:00–12:00 | Diapositiva 8. «En el laboratorio levantamos PostgreSQL con Docker y el chequeo `/api/health` respondió HTTP 200 con la base disponible. El frontend también corrió en un contenedor con Nginx. Esto nos permitió probar la comunicación entre partes reales del sistema. El equipo tiene un PR para integrar todos los componentes en Compose, además de trabajos de pgAdmin, backups y réplica. Antes de mostrar una réplica o reiniciar el stack completo, tenemos que verificar esa configuración. Para proteger los datos de la demo no vamos a borrar volúmenes ni hacer un failover improvisado».

Transición: «Con eso se ve el lado de aplicación y datos. Ahora repasamos cómo organizamos el desarrollo y sus medidas de seguridad».

4. Valentín Iriarte — 12:00 a 16:00 — Diapositivas 9 y 10

12:00–13:35 | Diapositiva 9. «Para trabajar en equipo usamos Jira para repartir y seguir tareas y GitHub para versionar cada cambio. No trabajamos directamente sobre `main`: hacemos una rama por tarea, commit, push y Pull Request hacia `develop`. Las revisiones y la CI permiten detectar errores antes de fusionar. Ya se fusionaron nuestros PR de URL de API, frontend Docker, documentación, consultas, seguridad de IA, n8n persistente y base de monitoreo. Los PR #29 del dashboard y #30 de Alertmanager se presentaron con CI exitosa y estaban pendientes de revisión al preparar estas diapositivas». **Acción:** abrir GitHub si hay Internet; si no, mostrar la diapositiva. No afirmar que #29/#30 ya están fusionados si no se comprobó ese cambio.

13:35–16:00 | Diapositiva 10. «También tomamos decisiones para no exponer credenciales en GitHub: se versionan plantillas `.env.example`, mientras los `.env` reales quedan fuera del repositorio. Grafana, Prometheus y Alertmanager se publican en localhost para el laboratorio, no como herramientas abiertas a Internet. La aplicación utiliza roles y mecanismos de autenticación, y se reforzó la validación del endpoint de IA. Todavía hay integraciones pendientes: el HTTPS público del túnel, Mantis, Nextcloud, la réplica/failover y el Compose final. Está bueno mostrar dónde llegamos y qué evidencia falta en vez de decir que todos los servicios están conectados. La idea del proyecto es lograr esa integración progresiva con pruebas reproducibles».

Transición: «Para cerrar vamos a mostrar qué estamos midiendo y cómo verificamos el envío y la resolución de alertas».

5. Tiziano — 16:00 a 20:00 — Diapositivas 11, 12, 13 y 14

16:00–17:15 | Diapositiva 11. «A Prometheus le configuramos exporters para servicios, base de datos y recursos; después armamos un dashboard en Grafana. En la captura el backend, el frontend y PostgreSQL tenían valor 1, o sea disponibles. También aparecieron 10 series de CPU de contenedores y gráficos de CPU y memoria. Esas estadísticas del sistema operativo corresponden principalmente al entorno Linux de Docker Desktop, no a toda la Mac». **Acción:** abrir `http://localhost:3001/d/gymhub-overview`; si falla, usar esta captura real.

17:15–18:55 | Diapositiva 12. «Para las alertas definimos ocho reglas en Prometheus, relacionadas con disponibilidad, exporters, CPU, memoria, disco y conexiones a PostgreSQL. Las ocho pasaron las validaciones y aparecían con estado `health: ok`. También verificamos que Prometheus detectara un Alertmanager activo. Después hicimos una prueba controlada: mandamos una alerta sintética directamente a Alertmanager y la vimos como `active`. Finalmente enviamos su resolución y la consulta confirmó que ya no estaba activa. Esto prueba la recepción y resolución de alertas en Alertmanager. **No** prueba todavía que una de las ocho reglas de Prometheus haya disparado ni una notificación a Mantis o n8n». **Acción:** abrir `http://localhost:9090/alerts`; después `http://localhost:9093/#/alerts`. Como la alerta de prueba ya se resolvió, usar la captura histórica. No es necesario crear otra.

18:55–19:38 | Diapositiva 13. «También dejamos n8n en Docker con su cuenta owner, autenticación de dos factores y datos persistentes: al recrear el contenedor, el workflow de prueba seguía disponible. El flujo completo que buscamos es alerta → n8n → incidencia en Mantis → notificación. Esa parte depende de conectar el webhook y de que el resto de servicios estén operativos; por eso no la marcamos como terminada».

19:38–20:00 | Diapositiva 14. «En resumen, probamos la aplicación con tres roles, una edición guardada en PostgreSQL, métricas reales en Grafana y recepción/resolución de una alerta en Alertmanager. Los siguientes pasos son completar el Compose, el monitoreo de la réplica y las integraciones del grupo. Muchas gracias».

Demostraciones adicionales, solo si el equipo confirma que funcionan

Este apartado cubre los criterios de la presentación que todavía dependen de otros responsables. No incorporar ninguno al relato como hecho sin una prueba real y capturas del entorno del día. Si una integración funciona, reemplazar **parte del bloque de 12:00–16:00** o reducir la explicación teórica para mantenerse en 20 minutos.

1. **Acceso HTTPS público (SCRUM-60).** Abrir la URL real suministrada por el equipo desde un segundo dispositivo, comprobar certificado válido y que solo publique el frontend/proxy; no enseñar token del túnel. Si no hay URL verificada, mostrar `localhost:8081` y decir que el acceso externo está pendiente.

2. **CI/CD con un commit.** Mostrar en GitHub el último Pull Request y su ejecución CI con resultado exitoso; no hacer commits improvisados durante la defensa. El flujo de ramas y revisión cuenta como evidencia, pero no reemplaza la automatización de despliegue si aún no existe.

3. **Alerta real → n8n → Mantis → aviso (SCRUM-76/70).** Solo si existe una ejecución guardada y validada, mostrar en este orden el alert generado por Prometheus, la ejecución de n8n, la incidencia real en Mantis y la notificación, ocultando tokens y datos privados. La alerta sintética directa a Alertmanager usada hasta hoy **no** prueba este encadenamiento.

4. **Failover de réplica (SCRUM-45–48).** Pedir al responsable una captura previa de rol primario/réplica, estado `streaming`, medición del lag y recuperación realizada en entorno preparado. Nunca detener o promover la base que mantiene la demo sin un plan confirmado de respaldo.

5. **Informe asistido por IA en Nextcloud (SCRUM-73/74/78).** Si el workflow de n8n está probado, abrir una ejecución real, el archivo generado y su ubicación en la carpeta de Nextcloud. Si no, explicarlo como integración prevista.

Regla: no sumar todas estas demostraciones al programa de veinte minutos. Elegir solo las que el equipo pueda garantizar, ensayarlas y ajustar los tiempos.

Plan B para la defensa (sin alterar datos)

Si falla…	Qué hacer en el momento
Internet / GitHub	Usar PPTX/PDF descargados; describir PRs #29/#30 como abiertos a la fecha del material.
Login o frontend	Mostrar las tres capturas reales ya incluidas en la presentación. No improvisar reinstalaciones.
API o PostgreSQL	Mostrar resultado SQL `id=1 / Prueba SCRUM-54.`; NO ejecutar `down -v` ni alterar volúmenes.
Grafana	Mostrar captura del dashboard con backend=1, frontend=1, PostgreSQL=1, 10 series.
Alertmanager	Mostrar captura de alerta sintética `active` y salida que comprueba resolución.
n8n / Mantis	Explicar qué está probado (n8n persistente) y qué depende de integraciones aún pendientes.
Falta de tiempo	Mantener una sola prueba en vivo (edición ya comprobada + SQL), resto con capturas; priorizar cierre claro.

Checklist de apertura antes de empezar

1. Conectar cargador, desactivar notificaciones y tener la presentación abierta **en la computadora de la demo**.

2. Abrir por separado: `http://localhost:8081/login`, `http://localhost:3001/d/gymhub-overview`, `http://localhost:9090/alerts` y `http://localhost:9093`.

3. Corroborar que frontend/backend responden y que DB está accesible; si alguna falla, **usar capturas**, no tocar Docker Compose antes de la exposición.

4. Tener la consola con el comando SQL *de solo lectura* ya copiado; nunca mostrar `.env`, tokens, contraseñas o 2FA.

5. Acordar quién avanza cada diapositiva y practicar el pase de palabra con cronómetro 4 min por integrante.

6. Opcional: si el equipo termina túnel, Mantis, Nextcloud, Compose o réplica, añadir evidencias **solo después de comprobarlas**; no cambiar el relato a último momento sin ensayo.

Preguntas típicas de la defensa (respuestas cortas)

¿Por qué Docker? Aísla servicios y facilita desplegar el mismo entorno sin reinstalar todo; en la demo aún hay un backend ejecutándose en el host.

¿Cómo saben que la rutina se guardó? Porque se editó desde la web y después PostgreSQL devolvió la fila `id=1` con el nuevo nombre.

¿Qué mide `probe_success=1`**?** Que el chequeo HTTP del servicio responde correctamente, no que todas sus funciones estén libres de errores.

¿Por qué no aparecen alertas en firing? Porque las ocho reglas estaban sanas e inactivas. Probamos Alertmanager con una alerta sintética, sin apagar servicios.

¿n8n ya crea incidencias en Mantis? No está demostrado; se probó que el entorno n8n es seguro y persistente, pero el flujo completo aún requiere integración.

¿Por qué no muestran la réplica? El equipo tiene cambios en revisión; no comprobamos el lag de la réplica en este entorno y no corresponde simular evidencia.

¿Qué pasa si falla Internet? La presentación y sus capturas funcionan sin Internet; el acceso a los servicios de laboratorio depende del equipo donde se hayan iniciado.

Referencias internas y evidencias

• Repositorio GymHub — `develop` y PR hacia esa rama.

• SCRUM-88 en Jira.

• SCRUM-67 PR #29 — dashboard Grafana.

• SCRUM-68 PR #30 — alertas y Alertmanager.

• Capturas integradas en `Presentacion_GymHub_HIA_2026.pptx` procedentes de las pruebas de noviembre de 2026.

• Las evidencias verificadas incluyeron login de tres roles, la consulta `rutinas` y las lecturas de Prometheus y Alertmanager.

Este documento describe el estado observado al prepararse para la exposición. Si algún PR cambia de estado, actualizar el texto antes de atribuir ese cambio a la demo.